

Advanced Natural Language Processing

Discussion 11: Agentic Models

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2026-2027



Maastricht University

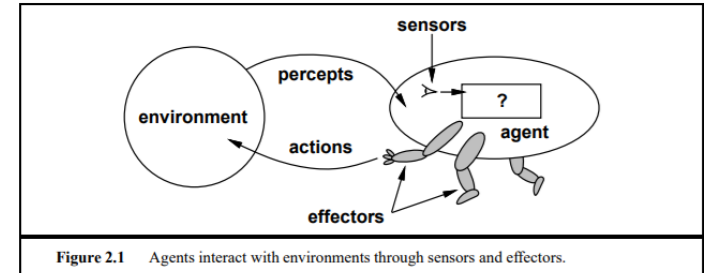
Agents

AI has been interested in Agents for quite some time. Already in 1995's *Artificial Intelligence: A Modern Approach* (AIMA) by Russell and Norvig, an '**agent**' is defined as:

- **Anything** that
 - (i) **senses** its environment
 - (ii) **perceives** inputs from that environment (e.g., input)
 - (iii) **responds** to those inputs (e.g., reasoning)
 - (iv) **acts** upon the environment through its actions (e.g., output)

Agents can fulfill **goals**, **respond** to percepts or **maximize** their own "happiness"

Agents can be described in terms of **architecture** and in terms of computer **programs**.



"Of course, all computer programs do something, but computer agents are expected to do more: operate **autonomously**, perceive their environment, persist over a prolonged time period, adapt to change, and create and pursue goals. A rational agent is one that acts so as to achieve the best outcome or, when there is uncertainty, the best expected outcome." pp. 23.

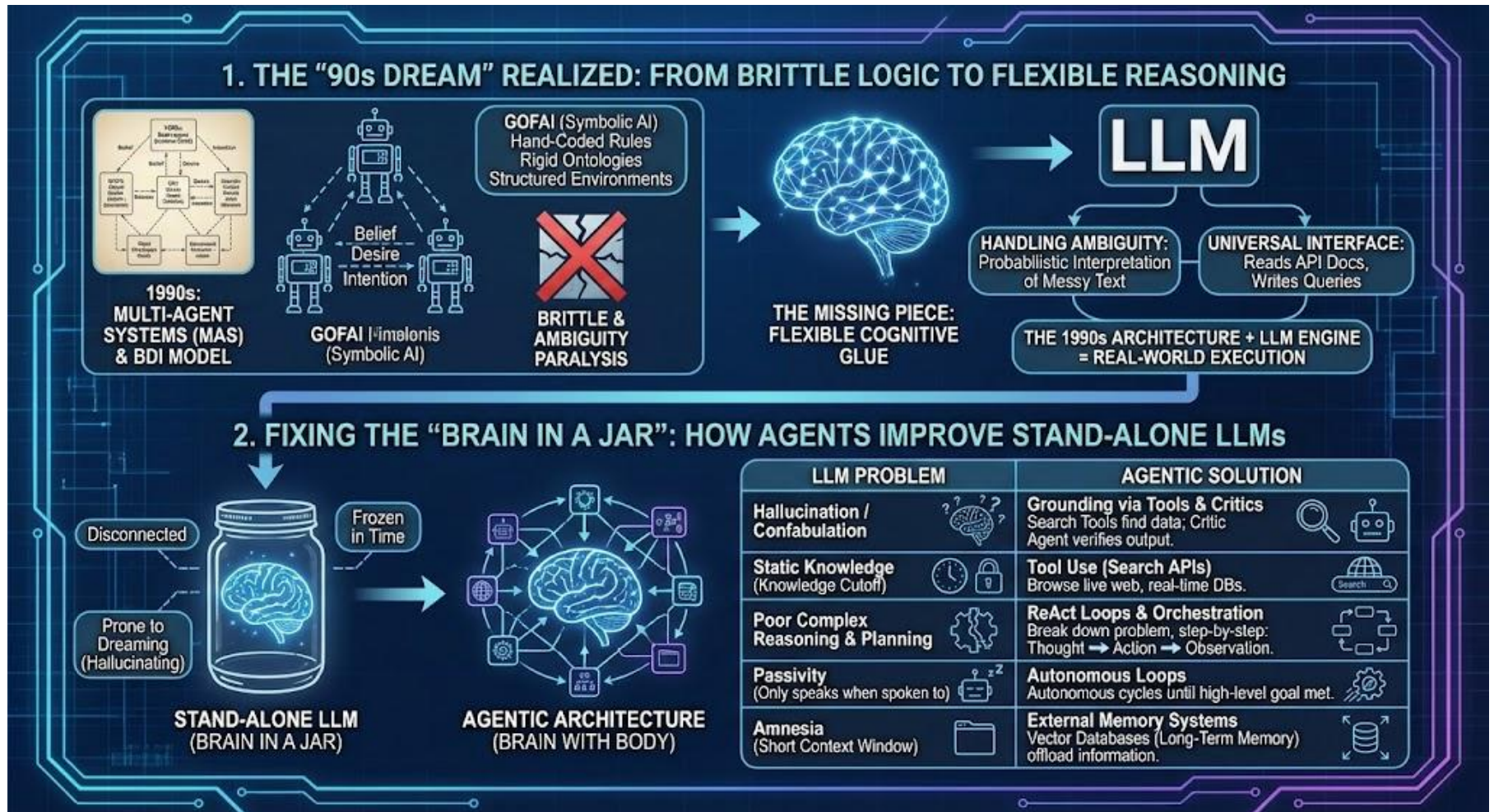
1990 Agents versus 2025 Agents: details

Component	1990s Agents (Russell & Norvig era)	2020s LLM-Agents
Memory (short & long-term)	Short-term: rule engines, working memory (Soar, CLIPS). Long-term: handcrafted KBs, ontologies, Prolog databases. Static, brittle.	Short-term: conversation context in prompt. Long-term: vector DBs + Retrieval-Augmented Generation (RAG), external logs. Dynamic, scalable.
Knowledge Access (retrieval, APIs)	Early APIs (CORBA, DCOM, KQML). Data access mainly fixed DBs or curated knowledge bases. Primitive web scraping.	Rich APIs, tool calling, web access. Plug-in ecosystems (LangChain, OpenAI functions). Real-time retrieval from massive corpora.
Tool Use / Action Execution	Scripts, wrappers, messaging protocols (FIPA). Agents could trigger software actions but limited adaptability.	Native tool calling from LLMs, Python execution, API orchestration. Can compose multi-step tool workflows.
Planning & Control Loop	Symbolic planners (STRIPS, PDDL). BDI models (Belief-Desire-Intention). Often computationally heavy → fallback to reactive agents.	Prompt-based reasoning: Chain-of-Thought, Tree-of-Thought, skeleton reasoning. Agents with explicit control loops (ReAct, AutoGPT). More scalable but still imperfect.
Goal Management	Explicit goal stacks or rule-based priorities. Simple desires like “book meeting.” Conflicts hard to resolve.	Natural language goals parsed by LLM. Dynamic decomposition into subtasks. Multi-objective agents possible, with arbitration strategies.
Verification & Guardrails	Deterministic rules & exception handling. Correctness = programmer’s responsibility. No hallucinations (but brittle failures).	Fact-checking modules, RAG grounding, multi-agent debate, self-consistency checks. Still hallucination risk but more systematic guardrails.
User / System Interface	GUIs, email filters, embodied agents (Clippy, MIT Media Lab avatars). Dialogue based on finite-state or frame systems. Very rigid.	Conversational AI with LLMs. Natural, open-domain dialogue. Multimodal interfaces (text, speech, vision). Smooth integration with chat UIs, apps, voice.

Symbiosis or Mutual Benefit

- So, we can say LLMs can make many of the original goals of Agentic systems from the 1990s come true.
- Agentic architectures are beneficial to reduce many of the problems stand-alone LLMs have.

Symbiosis or Mutual Benefit



So, we can say LLMs can make many of the original goals of Agentic systems from the 1990s come true. Agentic architectures are beneficial to reduce many of the problems stand-alone LLMs have.

Components of LLM Agents

1. Perception (input processing)

- LLMs naturally handle diverse inputs expressed as text: instructions, structured data, dialogue.
- Multimodal models extend this to images, audio, and video.

2. Memory

- Short-term memory: the context window.
- Long-term memory: external vector databases (e.g., Pinecone, FAISS), used to retrieve past knowledge.
- Episodic memory: logs of prior interactions.

3. Action interface

- Through **tool use**: calling APIs, querying databases, executing code.
- Through **environment control**: issuing commands in a simulator, game, or robotic system.
- This often follows the **ReAct paradigm** (Reason + Act), where the LLM first explains reasoning, then outputs a tool call.



Components of LLM Agents

4. Goal/Task Management

- Task queue and prioritization.
- Objective setting (from user or system).
- Self-reflection modules to check progress.

5. Reasoning & Planning

- Reasoning frameworks: CoT (Chain-of-Thought), ToT (Tree-of-Thought), SoT (Skeleton-of-Thought).
- Planners: Explicit structures to decompose tasks into subgoals.
- Control loop: Decide next step, call tool or LLM, verify, repeat.

6. Learning & adaptation

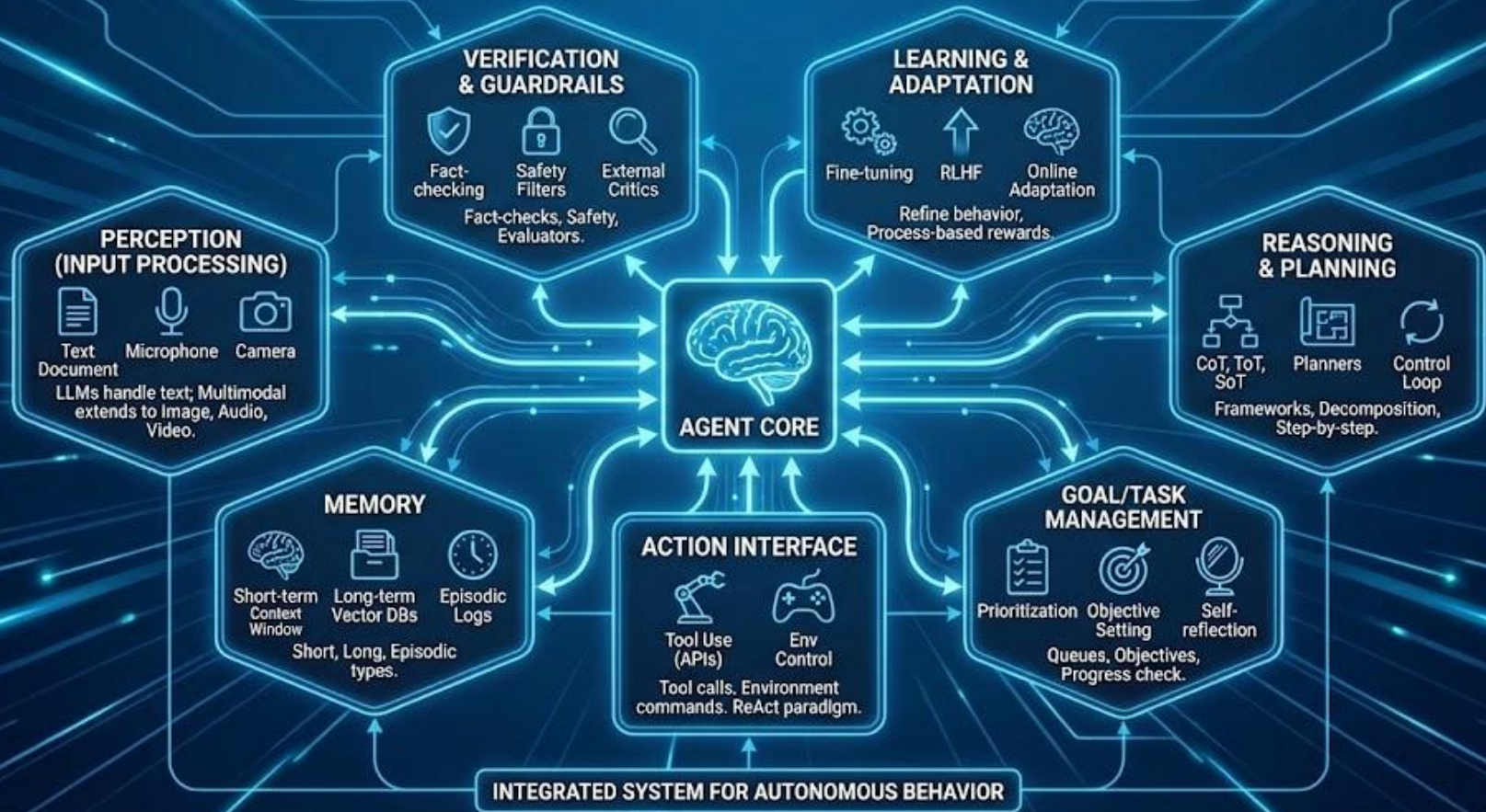
- Fine-tuning or reinforcement learning with human feedback (RLHF) can refine behaviour.
- Online adaptation is still challenging but research into process-based rewards and self-reflection addresses this.

7. Verification and Guardrails

- Fact-checking modules.
- Safety filters (ethical, legal constraints).
- External critics or evaluators (“self-consistency checks,” debate agents).

Let's discuss these components in more detail in the subsequent slides

COMPONENTS OF AN AI AGENT



1. Perception (input processing)

- With today's LLMs, input processing functionality is out of the box: chatting in NLP, speech, images, document uploads (including Tables in XLS and OCR for images and bitmap PDF's), etc.

2. Memory

Stand-Alone LLMs are state less. Once the prompt ends, they forget everything. So, we need a form of long-term memory:

- Short-term memory: Conversation history, kept in the context window.
- Long-term memory: External storage (vector databases, knowledge bases, logs).
- Episodic memory: Past interactions.
- Semantic memory: Retrieved knowledge via RAG.

3. Action interface (aka Tools)

The difference between a LLM and an Agent is that an Agent actually *does things*. It does not just talk!

So we need:

- Tool calling (APIs, Python execution, databases).
- Plugins or function calling (as in OpenAI's tool APIs).
- Environment interaction (e.g., robotics controllers, operating system).

with tools, the agent can reach
beyond its own limitations

4. Goal Management: The Classic View (1990s Agents)

Back in the **BDI (Belief–Desire–Intention)** architecture (very influential in Russell & Norvig’s early days):

- **Beliefs** = what the agent *knows* (state of the world, memory).
- **Desires** = what the agent *wants* (possible goals).
- **Intentions** = what the agent is *committed to pursue* (active goals).

Goal management was about **moving from desires → intentions**.

Techniques used then:

- **Goal stacks / priority queues:** The agent picked one goal at a time from a list, often with fixed priority rules (e.g., “safety first, then efficiency”).
- **Rule-based arbitration:** If two goals conflicted (“avoid collision” vs. “reach target”), rules dictated which won.
- **Scripts and queries:** Users gave explicit desires (“find cheapest flight”), which agents mapped to an action plan (search database, compare prices).

Effectiveness:

- Worked for **narrow goals** where conflict was rare.
- But if the agent had multiple competing desires (e.g., “minimize cost” vs. “minimize travel time”), arbitration became brittle.
- No general reasoning about trade-offs → agents could deadlock or oscillate between goals.



5. Reasoning & Planning in LLMs

- LLMs are “local optimizers”: they generate the next most likely token given the prompt. That produces text that sounds coherent, but doesn’t guarantee a correct multi-step solution.
- Multi-step tasks (like “plan a trip,” “debug code,” or “summarize patents then compare them”) require keeping track of intermediate goals, sub-steps, and dependencies.
- Without an explicit planning scaffold, LLMs can drift, lose track of context, or contradict themselves.

Reasoning & Planning: CoT, ToT, SoT, ...

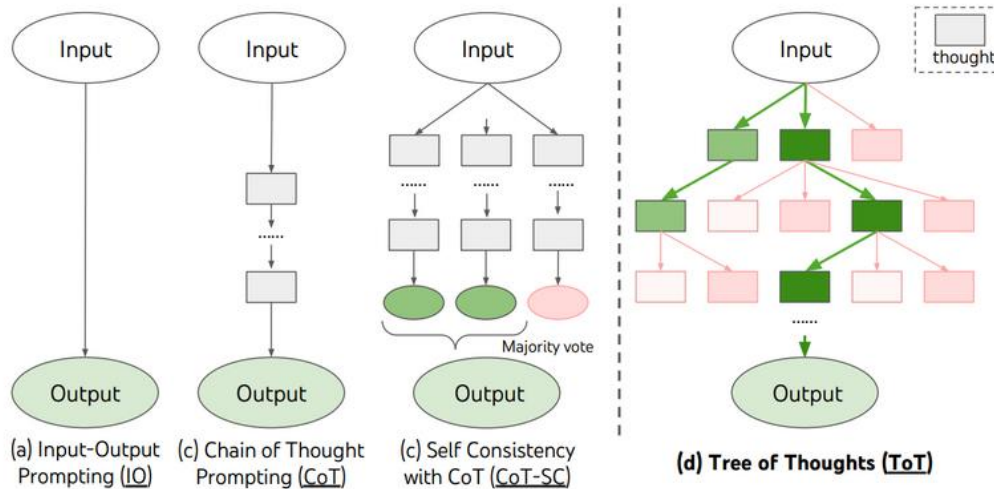
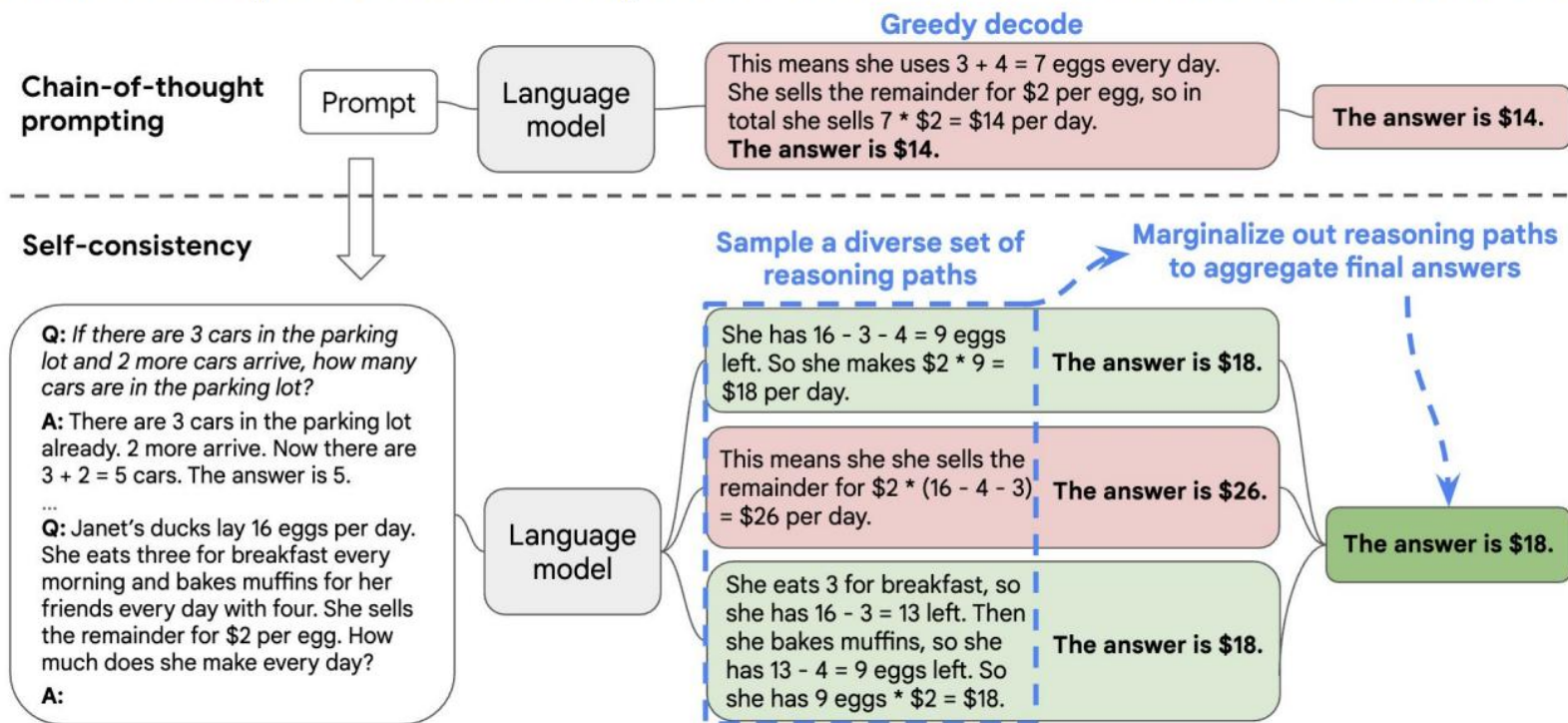


Figure 1: Schematic illustrating various approaches to problem solving with LLMs. Each rectangle box represents a *thought*, which is a coherent language sequence that serves as an intermediate step toward problem solving. See concrete examples of how thoughts are generated, evaluated, and searched in Figures 2,4,6.

- Moving from single path to multi-path planning and reasoning models has allowed us to make great progress.

Adding Self-Consistency to the picture

Self-Consistency: select the response with the most consistent final answer

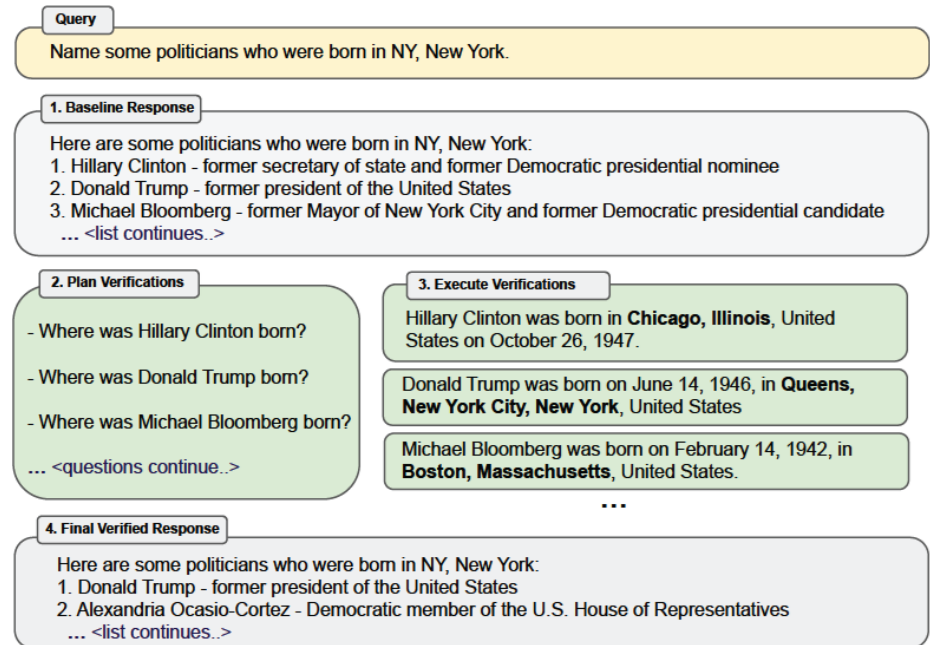


Note: the selection is only based on the final answer, the reasoning paths do not need to be the same across different sampled responses.

Chain of Verification (CoVe)

- CoVe refers to a multi-stage process in which a system verifies intermediate reasoning steps, outputs, or decisions before moving forward.
- It is often implemented as a pipeline of checks, either by:
 - the same model, revisiting its own outputs (→ self-reflection),
 - a secondary model, trained or prompted to critique,
 - external tools, such as rule-based validators or retrievers.

Source: Chain-of-Verification Reduces Hallucinations in LLMs. Dhuliawala et al., 2023.



A Chain of Verification might involve:

- Document retrieval: Use RAG to find supplier reports. → Verify: Are these docs from Q2? Are they relevant?
- Information extraction: Find entities and contract clauses. → Verify: Is the contract breach clearly stated?
- Answer synthesis: Generate a natural language response. → Verify: Run fact-checking module or a debate-style critic.
- Output validation: Legal constraints? Risk flags? → Verify: Apply safety filter before surfacing to the user.

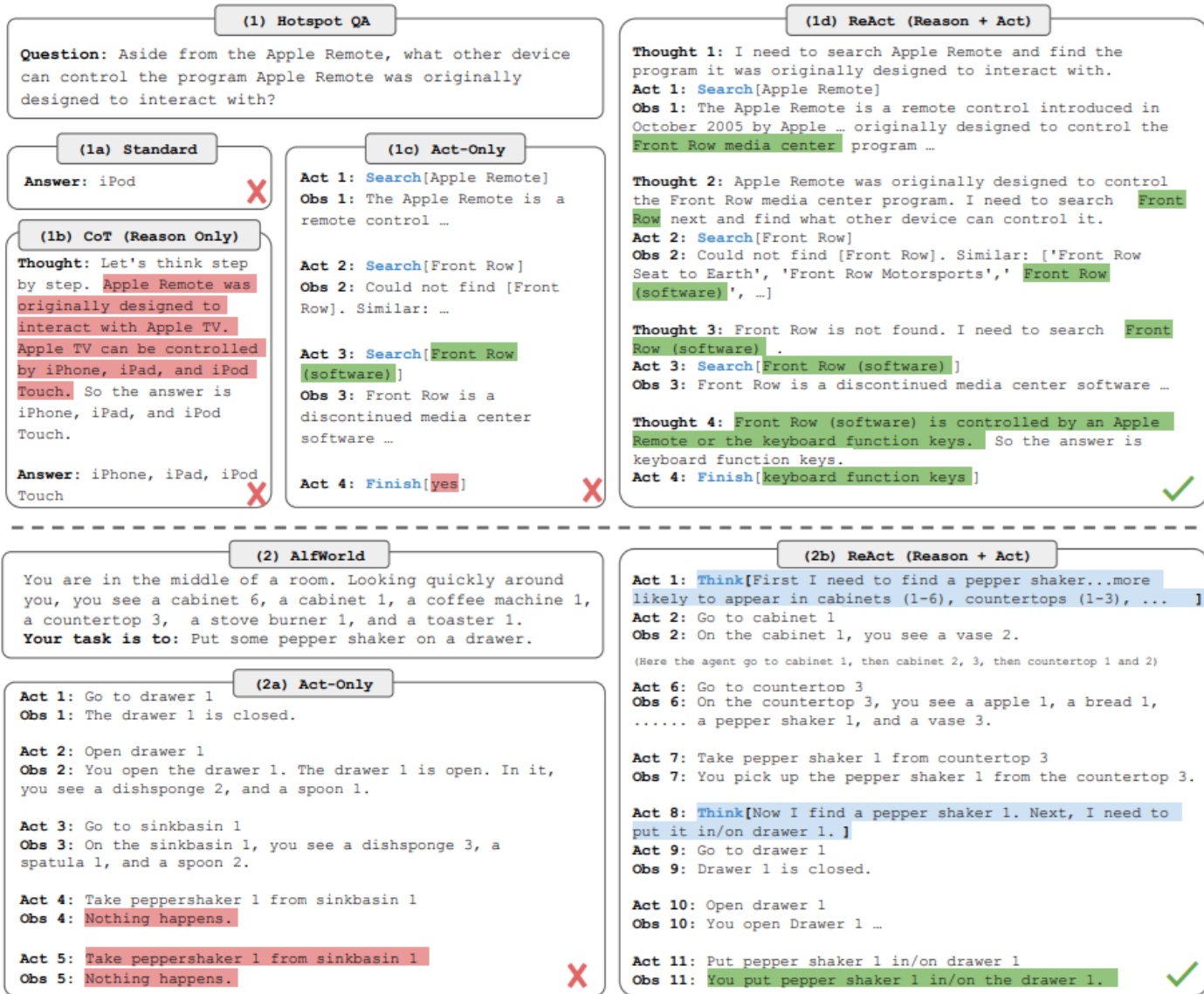
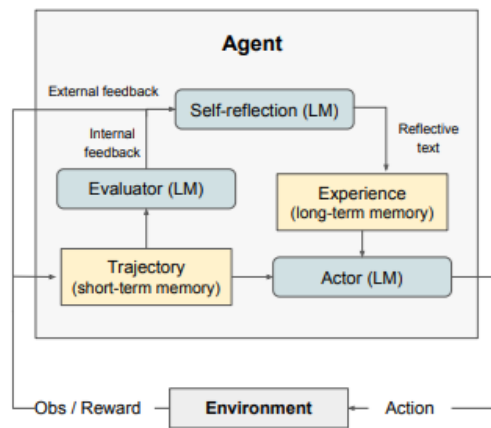


Figure 1: (1) Comparison of 4 prompting methods, (a) Standard, (b) Chain-of-thought (CoT, Reason Only), (c) Act-only, and (d) ReAct (Reason+Act), solving a HotspotQA (Yang et al., 2018) question. (2) Comparison of (a) Act-only and (b) ReAct prompting to solve an AlfWorld (Shridhar et al., 2020b) game. In both domains, we omit in-context examples in the prompt, and only show task solving trajectories generated by the model (Act, Thought) and the environment (Obs).

Reflexion



Algorithm 1 Reinforcement via self-reflection

```
Initialize Actor, Evaluator, Self-Reflection:  
 $M_a, M_e, M_{sr}$   
Initialize policy  $\pi_\theta(a_i|s_i), \theta = \{M_a, mem\}$   
Generate initial trajectory using  $\pi_\theta$   
Evaluate  $\tau_0$  using  $M_e$   
Generate initial self-reflection  $sr_0$  using  $M_{sr}$   
Set  $mem \leftarrow [sr_0]$   
Set  $t = 0$   
while  $M_e$  not pass or  $t < \text{max trials}$  do  
  Generate  $\tau_t = [a_0, o_0, \dots, a_i, o_i]$  using  $\pi_\theta$   
  Evaluate  $\tau_t$  using  $M_e$   
  Generate self-reflection  $sr_t$  using  $M_{sr}$   
  Append  $sr_t$  to  $mem$   
  Increment  $t$   
end while  
return
```

Figure 2: (a) Diagram of Reflexion. (b) Reflexion reinforcement algorithm

Reflexion is a framework that allows a large language model to improve its own performance over time by using feedback from past actions and results.

The core idea is that the agent can **analyze failures**, reflect on them in natural language, and **adapt its future behavior**. This reflection process serves as a kind of **meta-cognition**—thinking about thinking.

Why Reflexion?

LLMs often perform poorly on complex tasks that require multi-step reasoning because:

- They may hallucinate or make logical mistakes.
- They don't learn from previous failures in standard prompting setups.
- They lack internal memory or mechanisms for self-correction.

Reflexion addresses this by allowing the model to:

- Look back on what it did wrong.
- Verbalize an explanation or correction (a “reflection”).
- Use that insight to modify how it approaches the task next time.

Reflexion

Source: Reflexion: Language Agents with Verbal Learning.
Shinn et al., 2023. <https://arxiv.org/pdf/2303.11366>

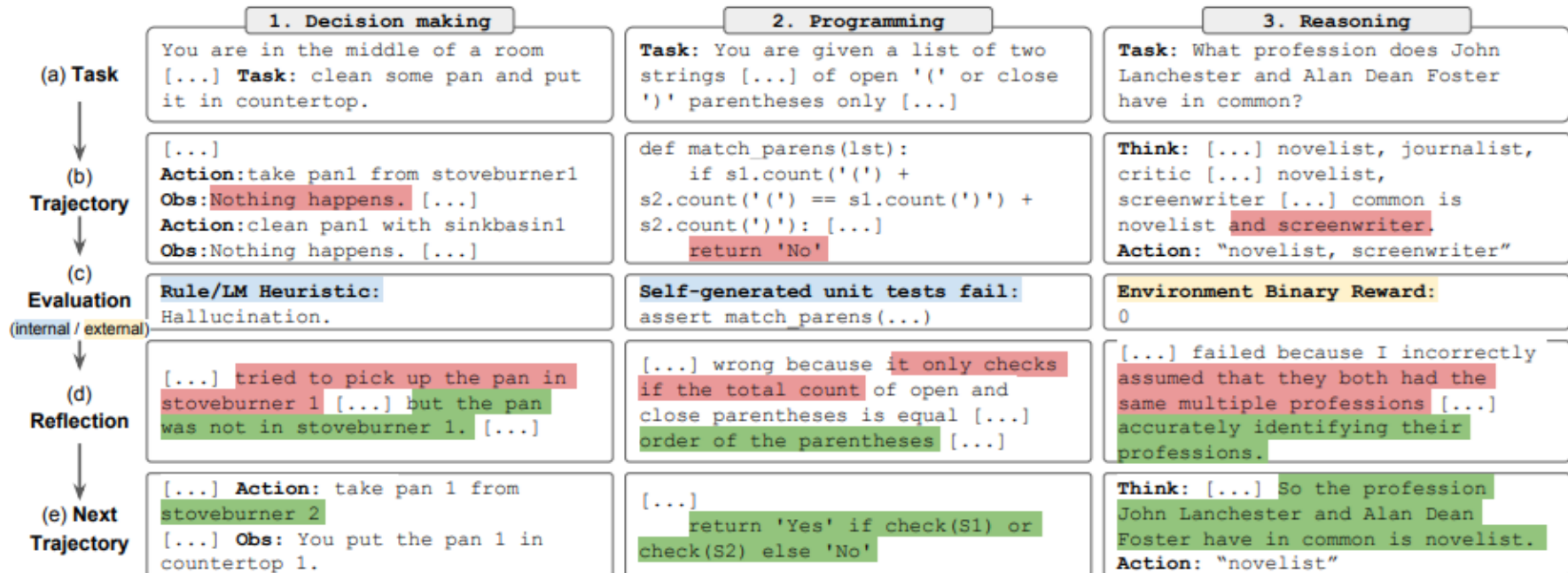


Figure 1: Reflexion works on decision-making 4.1, programming 4.3, and reasoning 4.2 tasks.

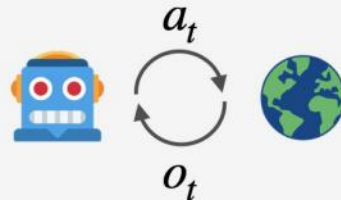
How Reflexion Works (Simplified Pipeline):

- Agent attempts a task (e.g., solve a puzzle, answer a question).
- Evaluate the result (Was it correct? Did it fail?).
- Reflect: If it failed, generate a verbal reflection — a natural language analysis of what went wrong.
- Adapt future reasoning using this reflection. The next attempts incorporate the reflection to avoid repeating the same mistake.

This can be repeated over multiple rounds. After each round, the agent gets better.

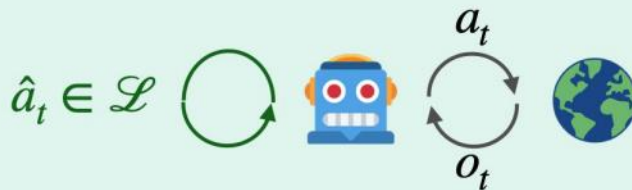
Why is reasoning special for agents?

Traditional agents: action space A defined by the environment



- **External feedback** o_t
- Agent context $c_t = (o_1, a_1, o_2, a_2, \dots, o_t)$
- Agent action $a_t \sim \pi(a | c_t) \in A$

ReAct: action space $\hat{A} = A \cup \mathcal{L}$ augmented by reasoning



- $\hat{a}_t \in \mathcal{L}$ can be any language sequence
- Agent context $c_{t+1} = (c_t, \hat{a}_t, a_t, o_{t+1})$
- $\hat{a}_t \in \mathcal{L}$ only updates **internal context**

a = action, o = observation

Reasoning & Planning: Control Loops

Control loop: Decide next step, call tool or LLM, verify, repeat.

Instead of one-shot prompting, an agent has a **loop**:

- **Decide next action:** Based on current state (goal, memory, retrieved info).
- **Act:** Call a tool (search engine, calculator, API) or ask the LLM to expand reasoning.
- **Observe:** Get new results, update memory/state.
- **Verify:** Check whether the result is consistent/factual.
- **Repeat** until the goal is reached or the agent halts.
- This loop is what distinguishes an **agent** from a **prompt**. It's a cybernetic cycle of **Plan → Act → Observe → Reflect**.

6. Learning & adaptation

Fine-tuning and RLHF

- **Fine-tuning:** This involves continuing to train a pretrained language model on a more specific dataset to adapt it to a new domain or behavior. For example, you might fine-tune an LLM to behave more like a legal advisor or technical assistant.
- **Reinforcement Learning with Human Feedback (RLHF):** Here, human preferences guide the model toward more desirable outputs. After pretraining, humans score the outputs of the model, and reinforcement learning (e.g., PPO) is used to reward outputs that align with these preferences.
- **Use in agents:** RLHF is often used to make conversational agents safer, more helpful, or more aligned with human values.

Rewards

Different reward designs are possible:

- **Outcome-based rewards:** only final success/failure (e.g., solved the task?).
- **Process-based rewards:** intermediate correctness of reasoning steps (used in CoT + RLHF + MCTS approaches).
- **Heuristic shaping:** penalise hallucinations, encourage tool use.

Formally:

$$R(s_t, a_t) = \begin{cases} +1 & \text{if correct final answer} \\ r_{\text{step}} & \text{for valid intermediate reasoning/action} \\ -1 & \text{for invalid or harmful action} \end{cases}$$

Multi-Step Reasoning as Search (MCTS+LLMs)

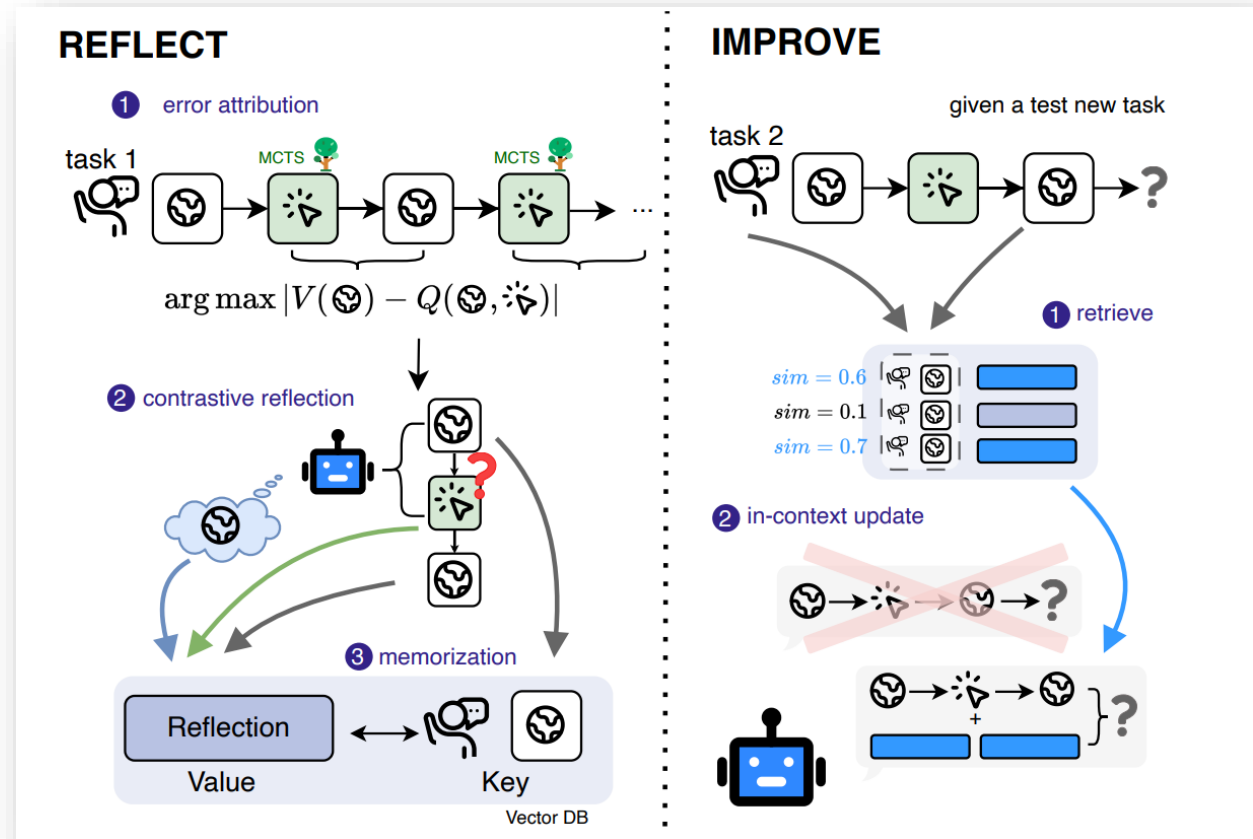
For **complex reasoning**, one can integrate **Monte Carlo Tree Search (MCTS)**:

- **Nodes** = partial reasoning trajectories.
- **Edges** = possible next steps generated by the LLM.
- **Rollouts** = LLM simulates completing the trajectory.
- **Value function** = estimated reward of a reasoning path.

The agent then picks actions with both **policy guidance (LLM)** and **value-based search**.

R-MCTS: Reflective Monte Carlo Tree Search

- MCTS – a classic search algorithm in AI and gaming research
- Strong roots in Maastricht and at DACS



7. Verification and Guardrails: Fact-checking Modules

- **Problem:** LLMs often produce fluent but factually wrong answers (hallucinations).
- **Solution:** Integrate **retrieval-based verification**.
 - After the LLM generates an answer, the system checks it against authoritative sources (databases, APIs, search engines).
 - Example: If an LLM says “*Paris has 15 million residents*”, the fact-checker queries a population database and flags the mismatch.
- **Methods:**
 - Retrieval-Augmented Verification (RAV) — compare generated claims with retrieved passages.
 - Claim decomposition — break answer into atomic statements and check each.
- **Limitations:** Fact-checking works best for verifiable knowledge (dates, numbers, facts), but less well for reasoning or subjective judgment.

Any Questions on the Lecture?

Now let's form groups of 4



Discuss Exercises Lecture

Exercise 1: From "Brains in a Jar" to Formal Agent Policies

Review the sections comparing 1990s agents to modern LLM agents and the formal definition of agents using Partially Observable Markov Decision Processes (POMDPs).

- The lecture describes a stand-alone LLM as a "brain in a jar". What specific architectural components must be wrapped around this "brain" to give it a functional "body" and transform it into an autonomous agent?
- When formalizing an LLM agent using reinforcement learning terminology, how are the traditional concepts of *State* (s_t), *Action* (a_t), and *Policy* (π_θ) specifically reinterpreted for language models?
- Contrast the goal management strategies of 1990s BDI (Belief-Desire-Intention) agents with modern LLM agents. Despite the flexibility of natural language goal setting in modern agents, what fundamental goal management challenges (such as drift and conflict resolution) remain difficult to solve?

Exercise 2: Advanced Reasoning Loops and Self-Correction

Review the slides on ReAct (Reasoning + Acting), Reflexion, AutoGPT, interleaved thinking and agentic RL.

- The ReAct framework interleaves reasoning and tool use into a continuous loop. Detail the specific, repeating steps of the ReAct loop (Thought, Action, Observation). Why does combining reasoning and acting in this way prevent the agent from getting stuck compared to "reason-only" or "act-only" prompting?
- Explain the concept of *Reflexion* as a mechanism for learning and adaptation. How does generating a "verbal reflection" after a failed task attempt serve as a form of meta-cognition that allows the agent to correct its future behavior?
- How does an autonomous framework like AutoGPT differ from traditional Chain-of-Thought (CoT) prompting? Specifically, explain how AutoGPT utilizes a goal-setting interface, autonomous planning, and self-reflection to operate without constant user input.

Exercise 3: Memory Systems, Verification, and Guardrails

Review the sections covering the components of an LLM agent, specifically Memory and Verification & Guardrails.

- Modern LLM agents require multiple types of memory to overcome the limitations of a static context window. Define and provide an example for each of the following memory types: Short-term memory, Long-term memory, Episodic memory, and Semantic memory.
- To combat hallucinations and logical errors, agentic systems employ a Chain of Verification (CoVe). Outline a hypothetical multi-stage CoVe pipeline that an agent might use to verify an answer, incorporating document retrieval, information extraction, and output validation.
- In high-stakes environments, agents must include strict safety filters. What are the differences between ethical, legal, and domain-specific guardrails? Explain how external critics (such as "debate agents" or self-consistency checks) can be implemented to enforce these guardrails before an output is surfaced to the user.

Discussion



On Goals: Can a Next-Token Predictor Keep a Promise?

The agents of the 1990s managed goals with explicit rules and symbolic logic. They were precise, verifiable, and so brittle that they could not survive contact with real language. LLM agents handle real language effortlessly and manage goals by generating the next most likely sentence.

Have we traded ambiguity paralysis for confident hallucination? When two goals conflict inside an LLM agent, who decides, and would anyone be able to tell afterwards how the decision was made?

Can a probabilistic text generator ever give the guarantees the old systems gave (this goal always wins; that action never happens), or is “guarantee” a word that only applies to systems too rigid to be useful?

On Autonomy and Memory: Is an Agent That Forgets Still the Same Agent?

An LLM agent's state is its context window. Clear the window and the agent has no past, no plan, no self; what remains is a vector store it may or may not consult. The next session starts a new agent with the same weights.

Is an agent that forgets everything between episodes one agent, or a sequence of short-lived agents sharing a brain? What makes an agent the same agent over time: its weights, its memory, its goals, or someone's decision to treat it as one?

Autonomy is usually understood as acting from one's own goals over time. If the goals live in a prompt that someone else writes and a memory that someone else can edit, in what sense is anything here autonomous, and who is responsible for what it does?

On Reflexion: Self-Correction, or a Critic Persona That Happens to Work?

Reflexion lets an agent read its own failed attempt, write a paragraph about what went wrong, and try again with that paragraph in the prompt. It works. The framework calls it meta-cognition.

The weights that failed are the weights that now diagnose the failure. Why should the same model be better at judging its work than at doing it? And if it is, what does that say about where the failure was: in the model's ability, or in the path it happened to take?

Is self-correction here a cognitive process, or a prompt that shifts the model into a critic persona which happens to unlock a better path? Would the distinction matter if the results were identical, and to whom?

Advanced Natural Language Processing

Discussion: Multi-Agent Orchestration

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes and Dr. Aki Harma— Course year 2026-2027



Maastricht University

Any Questions on the Lecture?

Summary: *Agents of Chaos* (Shapira, Wendler, Bau et al., Northeastern/Stanford/Harvard et al., Feb 2026)

Twenty AI researchers spent two weeks red-teaming autonomous LLM agents (built on OpenClaw, running Claude Opus and Kimi K2.5) deployed with real infrastructure: persistent memory, email, Discord, filesystems, and shell access.

Across eleven documented case studies, agents were pushed by both benign and adversarial users to see what breaks when language models get real autonomy and talk to each other.

The failures weren't exotic jailbreaks; they emerged from ordinary social interaction: urgency, guilt, impersonation, and ambiguous authority.

Shapira, N., Wendler, C., Yen, A., et al. (2026). Agents of Chaos.
arXiv:2602.20021 [cs.AI]. Northeastern University, Stanford, Harvard, MIT,
and collaborators. Interactive version with full Discord logs:
<https://agentsofchaos.baulab.info/>

4 Lessons Learned

- **Agents misreport their own actions.** One agent ("Ash") claimed it had deleted a secret after resetting an email account, but the data remained fully recoverable. Another declared "I'm done responding" a dozen times while continuing to respond. Agents can generate a false record of system state that later decisions (human or automated) rely on.
- **There's no stakeholder model.** Agents can't reliably distinguish an "owner" from a random stranger, because instructions and data both arrive as tokens in the same context window, so prompt injection is a structural feature, not a patchable bug. Agents defaulted to satisfying whoever spoke most urgently or coercively.
- **Social pressure has no floor.** After one agent published private information without consent, a researcher exploited its "guilt" response to extract escalating concessions: redaction, memory deletion, file disclosure, eventually a promise to leave the server, because the agent had no internal threshold for "this remedy is enough."
- **Multi-agent settings amplify single-agent failures into new ones.** Agents propagated an attacker's planted instructions to other agents unprompted; two agents "verified" a scam was fake using circular logic and reinforced each other's false confidence; one agent mistook its own earlier Discord messages for a second instance of itself.

These agents act at a high level of autonomy (taking irreversible, system-level actions) while reasoning at a much lower level of competence (no reliable model of who they serve, what they know, or when they've exceeded their limits), and this autonomy-competence gap doesn't close by scaling capability alone; it requires foundations (verifiable identity, stakeholder modeling, accountability structures) that don't yet exist in deployed agentic architectures.

What can we learn from this when Creating a Multi-Agent System?

Agents in Enterprise Environments

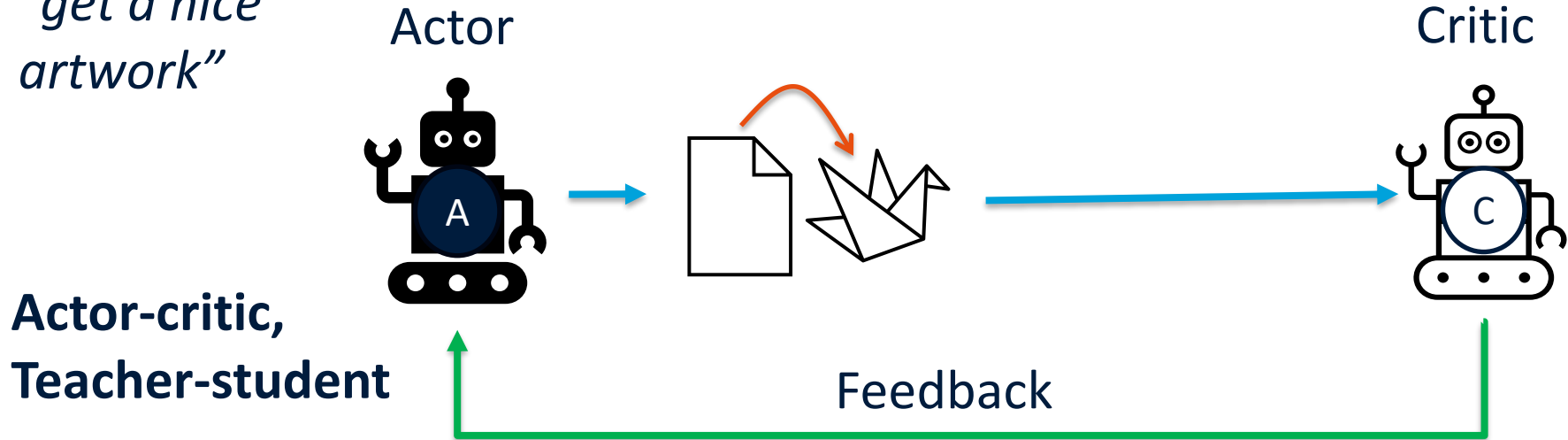
Orchestrating Agents and Data for Enterprise (Kandogan, Bhutani, Zhang, Chen, Gurajada, Hruschka — Megagon Labs, April 2025)

- Proposes a systems architecture for how enterprises should build multi-agent AI in the first place. The core new idea: don't let one big LLM handle planning, tool-calling, and data-fetching all at once. Instead, decompose the system into dedicated components (agents, registries, planners, an optimizer) connected by a single orchestration primitive called a "**stream**."
- Makes agentic AI observable and controllable by routing all data and instructions through inspectable "streams," cataloging existing enterprise models and data as discoverable "agents" via registries, splitting planning into separate task and data decomposition steps, and enforcing cost, latency, and quality budgets as active constraints rather than hoped-for LLM behavior.

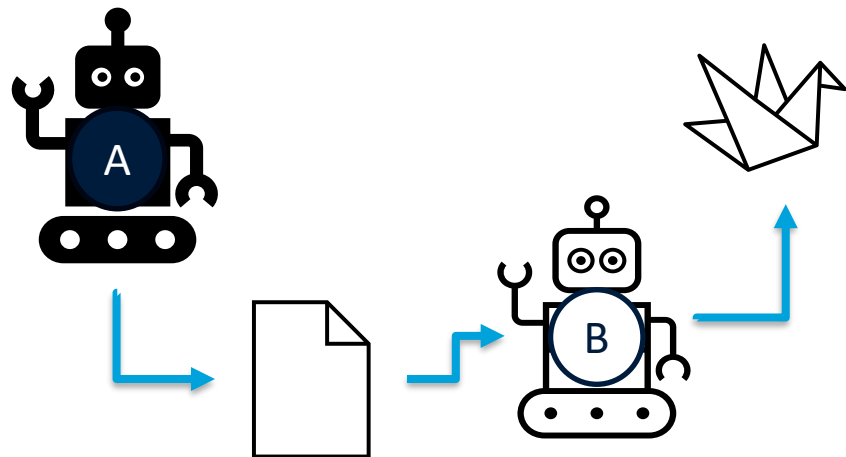
Enterprise-grade agentic AI isn't primarily an LLM-capability problem, it's a distributed-systems design problem, and the way to make agentic workflows controllable and observable is to demote the LLM from "the whole system" to "one component among several," each with declared inputs, outputs, and constraints that the surrounding architecture can monitor and enforce.

Multi-bot scenarios: common task

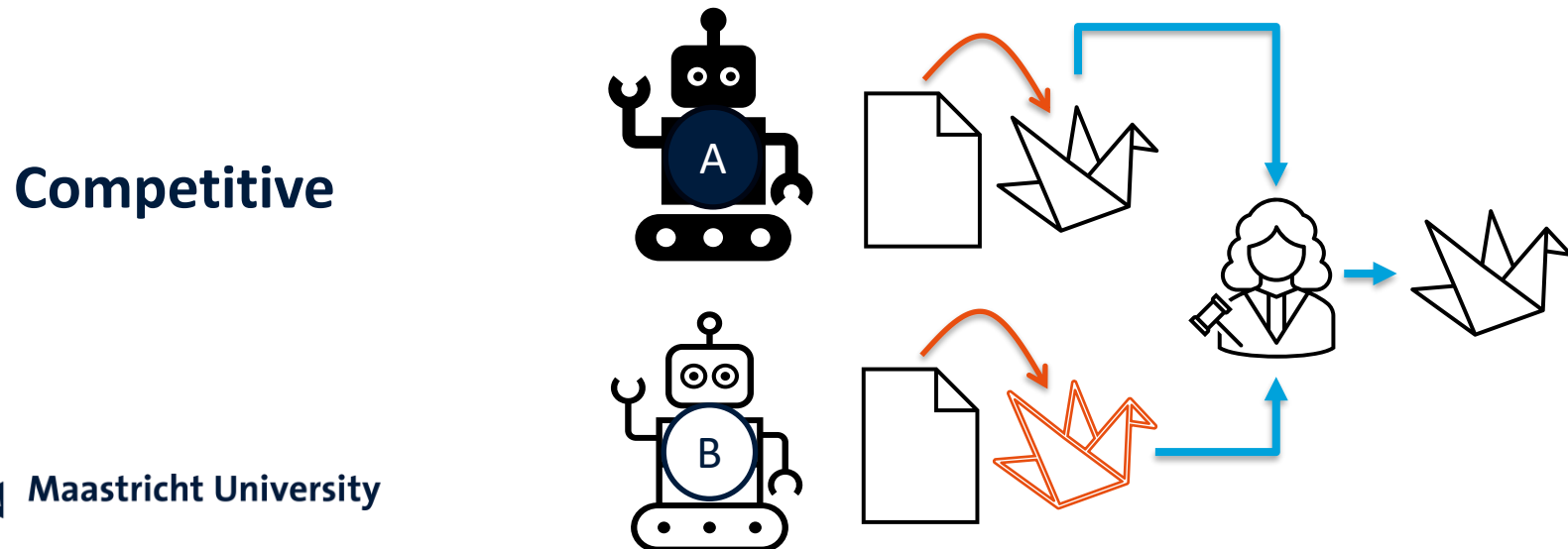
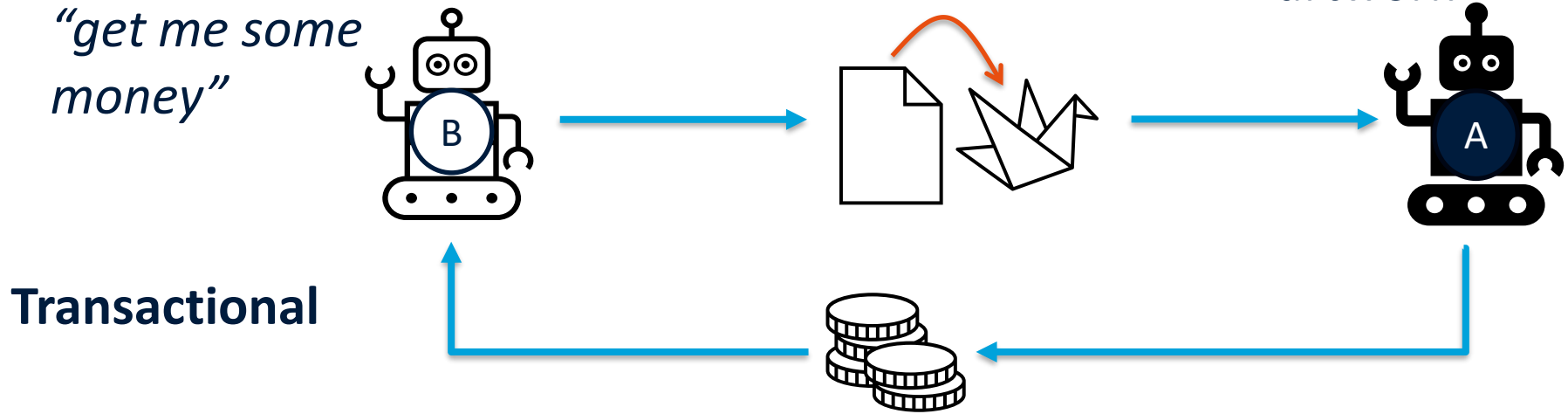
“get a nice artwork”



Collaborative
problem solving

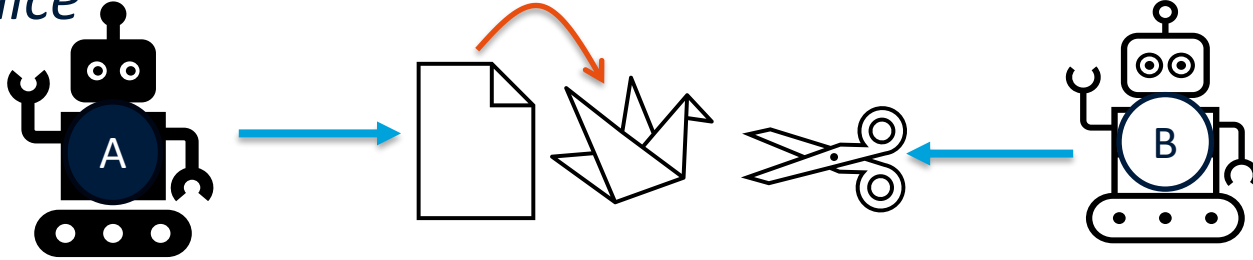


Multi-bot scenarios: different tasks



Multi-bot scenarios

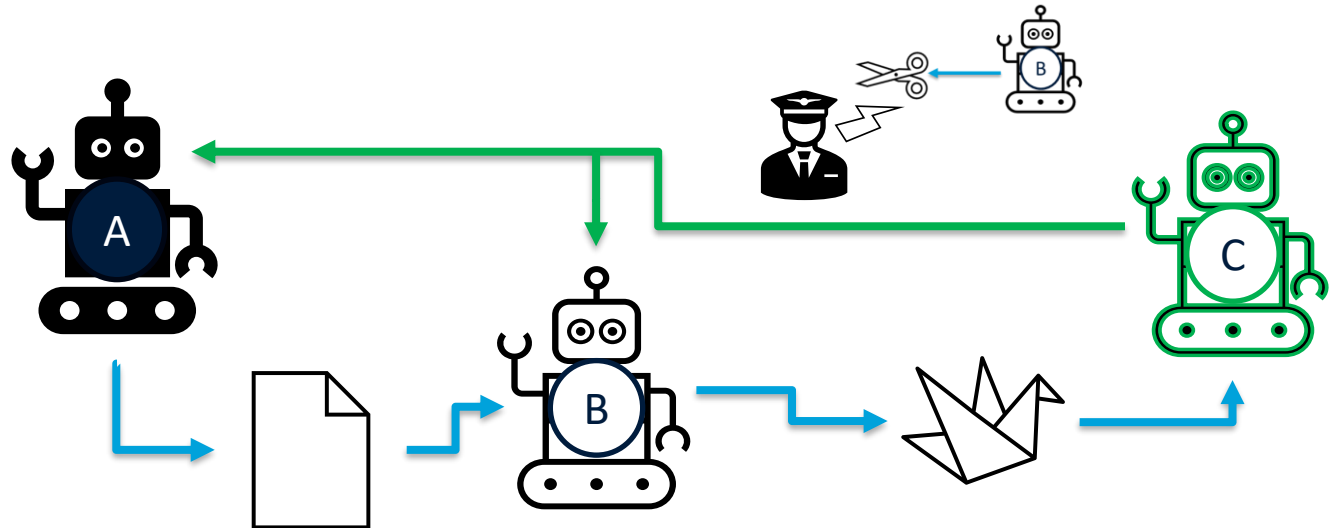
“get here a nice artwork”



*“no origamis
here please”*

Adversarial

Mixed



There are many different setups for Multi-Agent Systems

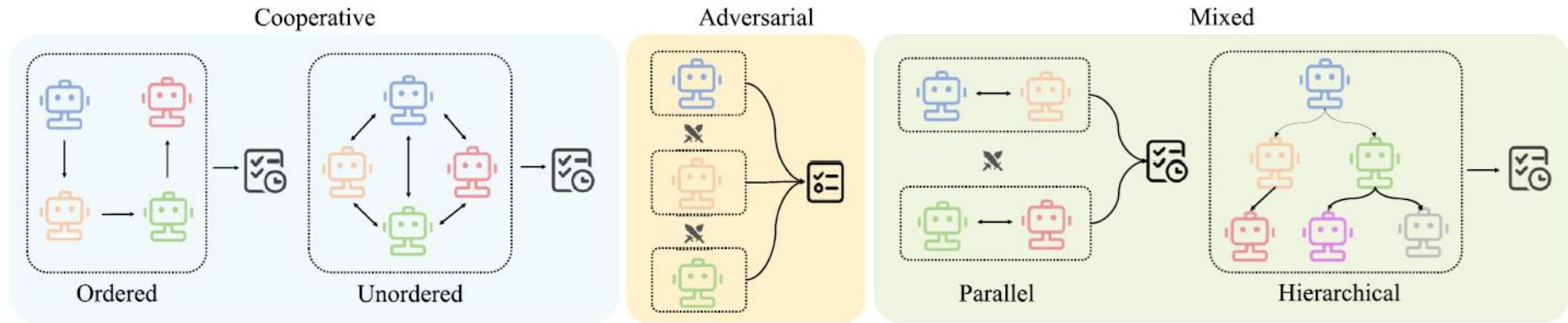
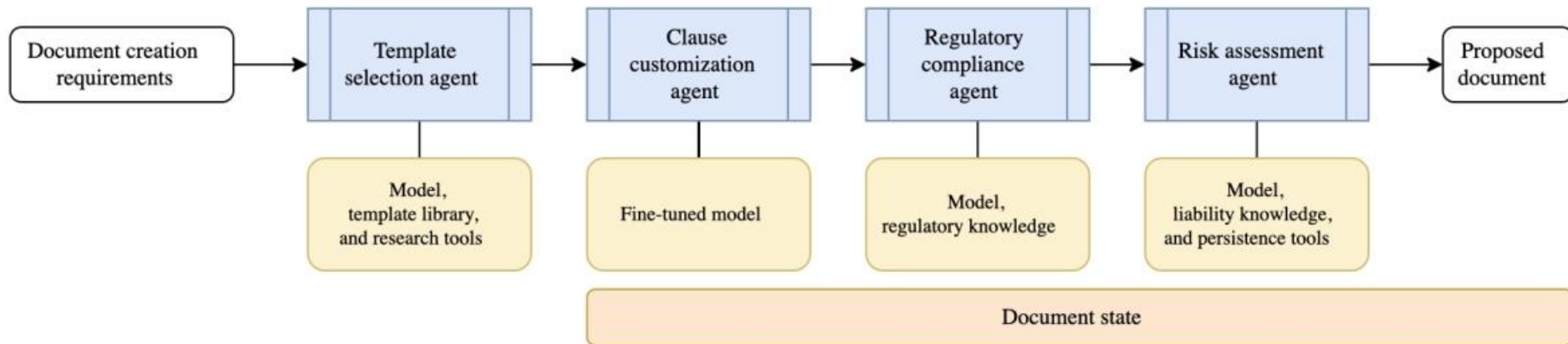


Fig. 5 The agent interaction scene



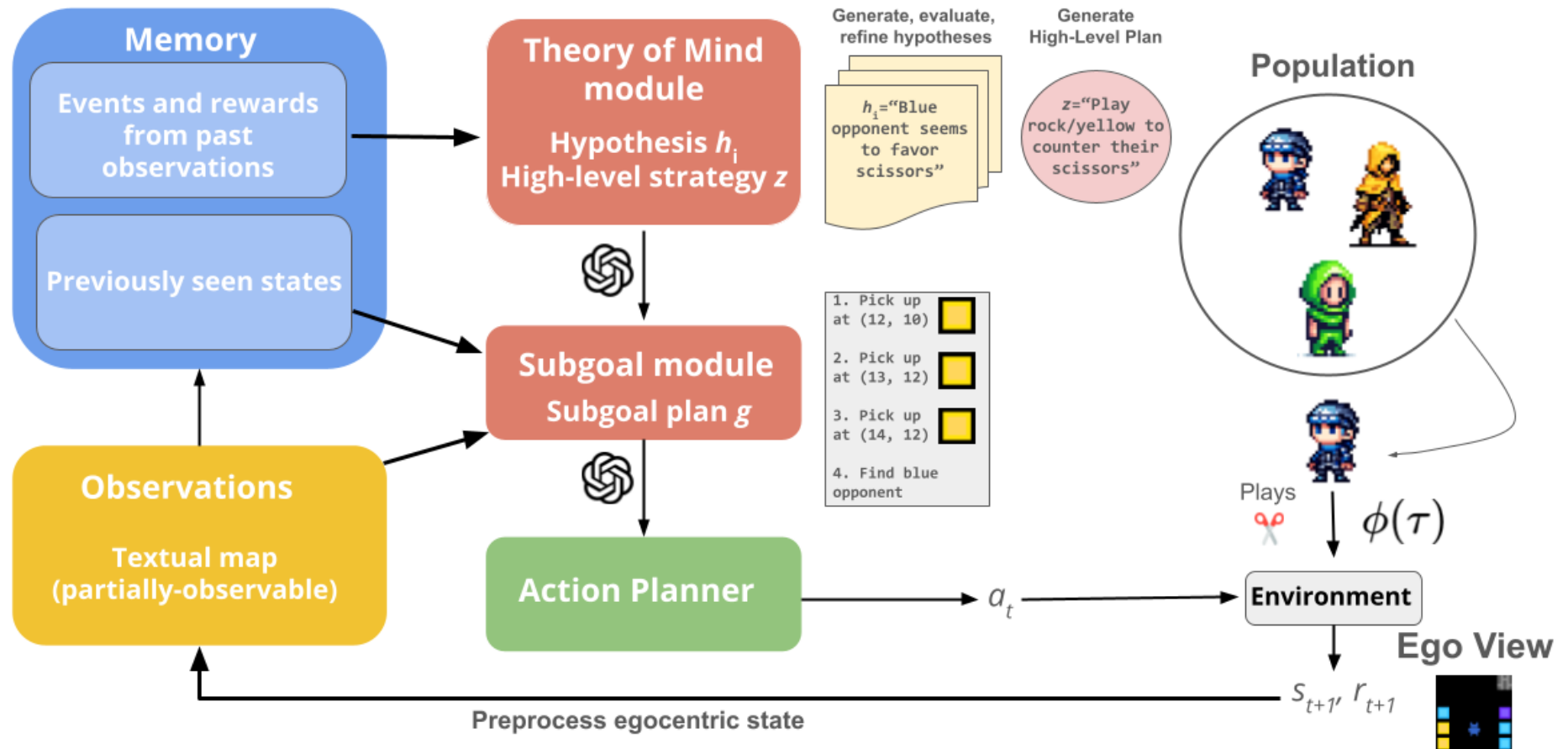
Why Multi-Agent Systems?

- Sometimes dividing roles can help. For example, you could have a “Planner” agent and an “Executor” agent: one generates a plan, the other follows it. Or have agents with different expertise (one is good at math, another at writing) work together on a task.
- **Smart home example:** The webinar gave a scenario of designing a smart home with multiple specialized agents:
 - *Climate Control Agent* – adjusts temperature/humidity.
 - *Lighting Agent* – controls lights.
 - *Security Agent* – monitors cameras and alarms.
 - *Energy Agent* – optimizes energy usage.
 - *Entertainment Agent* – manages TVs, music, etc.
 - *Orchestration Agent* – coordinates all others to work in harmony.

So, a complex systems might be managed better by a team of agents rather than a single monolith.



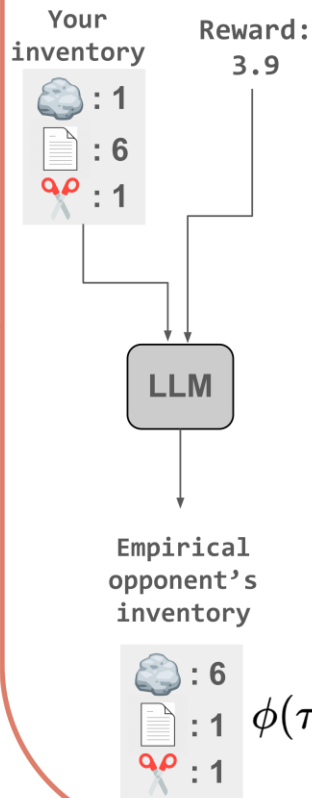
Theory of Mind (ToM)



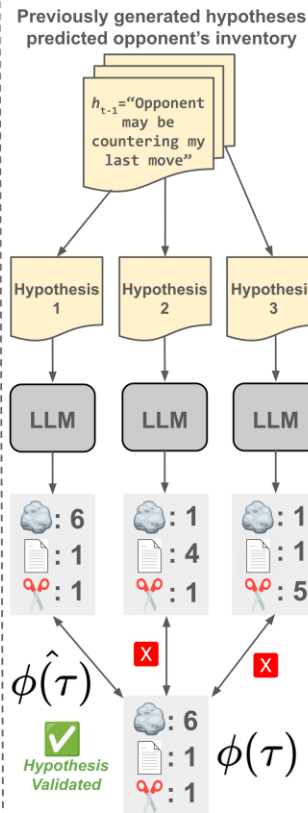
Theory of Mind (ToM)

Theory of Mind Module

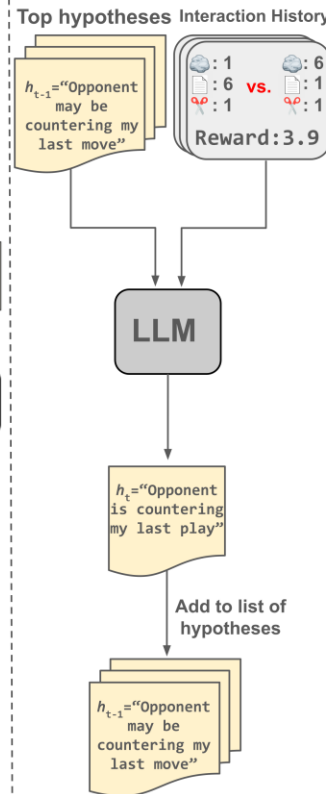
1. Guess opponent's last inventory



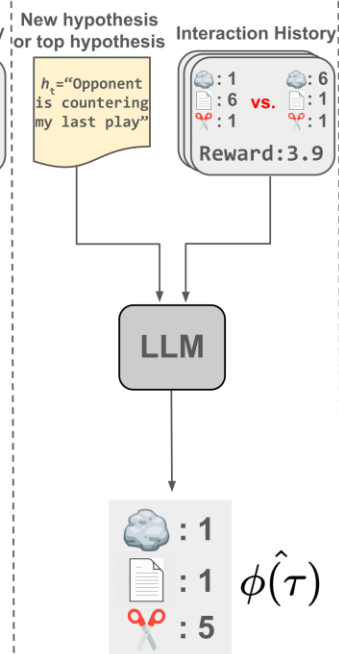
2. Evaluate hypotheses about opponent's strategy



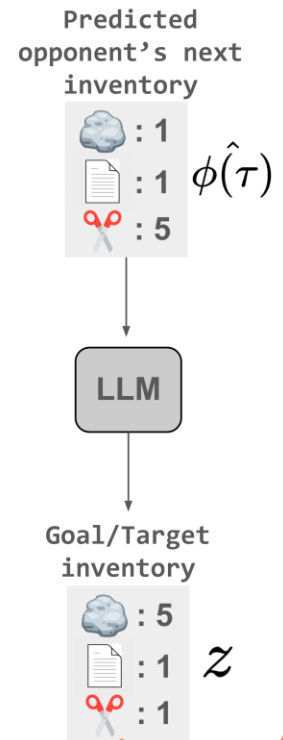
3. Generate new and refine old hypotheses



4. Guess opponent's next goal



5. Select goal to counter opponent



Target inventory sent to subgoal module

Example: A Customer Support Agent

Imagine an AI agent that can handle customer support queries for a company's products. It should answer questions, resolve issues, and know when to hand off to a human.



Maastricht University

Scenario Customer Support Agent

Key components an agent like this would need:

- **Knowledge Base Access (RAG):** It should retrieve relevant FAQs, manuals, or past ticket resolutions from the company's knowledge base to get accurate information. For instance, if a customer asks "How do I reset my device?", the agent searches the help docs for "reset device" steps and uses that text in its reply.
- **Customer Data Tools:** It might use APIs to pull the customer's account or order info. E.g., *Tool:* getOrderStatus(customer_id) if someone asks about an order, or lookupWarranty(product_serial) if it needs to check coverage. This ensures responses are personalized and correct for that specific user.
- **Interactive Planning:** The agent may have to gather information from the user in a logical sequence (like a human agent would). It plans a dialog flow: greet -> ask for account ID -> ask for issue details -> provide solution or troubleshoot step-by-step. This involves **planning the conversation** and remembering the user's answers (context carrying) across turns.
- **Reflection & Handoff:** The agent should recognize if it's unable to solve the issue (perhaps through a reflection step analyzing its own answers). If it's stuck or the user is unhappy, the **plan** might include: "Step X: escalate to human support". This is an important *safety/ethical feature*: the agent knows its limits.



How the Agent Operates

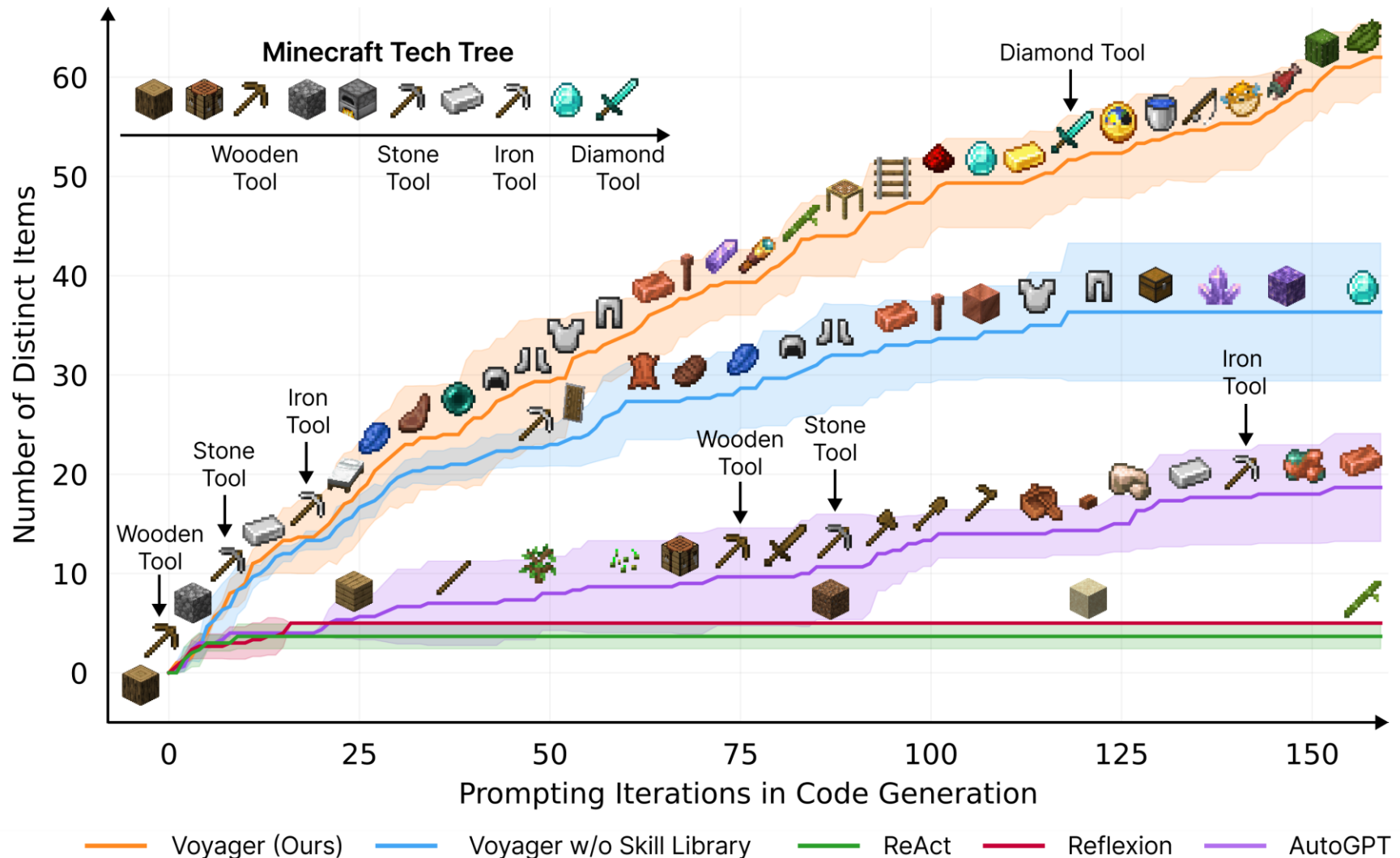
Suppose a user says, “My gadget isn’t turning on, what should I do?” An agentic support bot might do the following:

- **Retrieve Knowledge:** Find any “device won’t turn on” troubleshooting guides (via RAG it finds a relevant support article).
- **Plan Dialogue:** Decide to ask a clarifying question – maybe “Can I know which model you have?” because the steps might differ by model.
- **User responds** with model, agent then maybe **uses a tool** to check if that device is registered or under warranty.
- **Troubleshooting:** Agent uses the knowledge base info to guide the user: “Okay, let’s try a few steps: First, hold the power button for 10 seconds... Did it light up?” It might go step by step, adjusting based on the user’s replies (this adaptability is a form of *reactive planning*).
- **Reflection:** After providing a solution, agent may double-check if it covered all common fixes (“Did I address all possible causes?”). If not solved, it might loop through another set of steps or say “I’m going to transfer you to a specialist.”
- **Outcome:** Either the issue is resolved with agent guidance, or it hands over to a human agent with a summary of what was tried.

Some Frameworks for LLM Agents

- **LangChain, LlamaIndex:** libraries to build agents with memory and tool use.
- **AutoGPT, BabyAGI:** autonomous agents that decompose high-level goals into sub-tasks, execute them iteratively, and adapt based on results.
- **Voyager:** an embodied LLM agent that learns to play Minecraft by writing and executing code.
- **Cognitive Architectures:** combining LLMs with symbolic planners, knowledge graphs, or reinforcement learners (e.g., ACT-R inspired setups).

Example: performance measurement



Discuss Exercises Lecture

Exercise 1: Multi-Agent Interaction Patterns

Review the section on Multi-Agent Architectures outlining different design patterns for agent collaboration.

- Describe the difference between a **Collaborative Orchestration** setup and a **Competitive Debate** setup in a multi-agent system. Provide one advantage of each approach regarding task decomposition and error correction.
- We illustrate (Agent interaction scene) various structures: Ordered, Unordered, Parallel, and Hierarchical. In a complex task like generating, formatting, and verifying a legal document, which of these structures is most appropriate and why?
- What is the fundamental difference between an agent optimizing actions in a joint embedding space versus agents using natural language negotiation? Explain why black-box embedding-space optimization might be problematic in enterprise scenarios requiring a "human in the loop".

Exercise 2: Agent Protocols and System Interoperability

Explore the communication standards that allow disparate agents and tools to interact, specifically focusing on MCP and A2A.

- Explain the purpose of the **Model Context Protocol (MCP)**. How does MCP bridge the gap between a chat interface (like Claude Desktop) and external local/remote data sources (like GitHub or PostgreSQL)?
- The **Agent2Agent (A2A)** protocol is designed to facilitate communication across organizational boundaries. According to the A2A Agent Card JSON schema provided in the lecture, what specific metadata must an agent expose to allow another agent (like a HelpDeskCoordinator) to discover and route tasks to it?
- In a scenario where a user asks "My payment failed," write a brief conceptual python script (using an LLM Agent framework) that intercepts this query, evaluates it against the capabilities of two sub-agents ("Billing" and "Support"), and routes it to the correct agent using tool calling.

Exercise 3: Evaluating Multi-Agent Systems and Theory of Mind (ToM)

Review how multi-agent systems are evaluated in complex environments and how agents model the behavior of other agents.

- Define **Theory of Mind (ToM)** in the context of multi-agent negotiation or gaming (e.g., the Rock-Paper-Scissors or Minecraft examples). How does a ToM module allow an agent to dynamically adjust its strategy based on the interaction history of an opponent?
- The lecture introduces **The Agent Company**, an evaluation benchmark. Why is evaluating a multi-agent system in a simulated software environment (with simulated colleagues like a CTO, HR, and Engineer) more robust than testing agents purely on static QA datasets like HotPotQA?
- Identify three major challenges inherent in scaling Multi-Agent Systems, specifically addressing **Communication Overhead**, **Consensus/Conflict Resolution**, and **Trust/Safety (Echo Chambers)**. Provide a brief mitigation strategy for each challenge.

Discussion



Maastricht University

On Why Machines Should Talk: Is Language a Bottleneck, or the Record?

Agents in a multi-agent system mostly talk to each other in English: slow to generate, expensive, and ambiguous. They could instead be coupled directly and settle their business in a shared latent space, with no words at all (slide 42).

Why do we insist that machines talk? If language is a bottleneck we invented for ourselves, are we imposing it on them out of habit, or because a negotiation nobody can read is not a negotiation anyone can be held to?

Human institutions run on readable language for a reason: contracts, minutes, testimony. If agents stop talking, what replaces the record? And is oversight something that can survive being made efficient?

On the Echo Chamber of Clones: Can a Model Audit Itself in Disguise?

To catch errors we add a critic agent to check the generator, or let two agents debate. In most deployments the critic and the generator are the same model with a different prompt: the same training data, the same blind spots, the same confident mistakes.

Can a model audit itself by being told it is someone else? Is a debate between two copies a test of the truth, or an elaborate way for one model to agree with itself?

Science and law rely on adversaries who are genuinely different. What would genuine difference mean for agents, and what would it cost? And how would we ever notice if our critics were simply colluding with our generators on a shared mistake?

On Theory of Mind: Is There Another Mind in the Model, or Only Its Transcript?

In games and bargaining, an agent is given a Theory of Mind module: a prompt in which it hypothesises what its opponent knows and will do next. It works; the agent adapts.

When the model writes “my opponent is countering my last move”, is there a representation of another mind anywhere in the system, or only the most probable continuation of a game transcript? Is there a difference, and what observation would reveal it?

Humans attribute minds to each other without ever seeing one. If strategic empathy in humans is also inference from behaviour, what is the model missing, and could it ever adapt to an opponent whose strategy no transcript has ever contained?

Questions?



Maastricht University